[Your College Logo]

Industrial Internship Report on

”Content Management System for a blog”

Prepared by

[Karuna Sri Peduri]

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. My project was (Tell about ur Project) This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

Preface

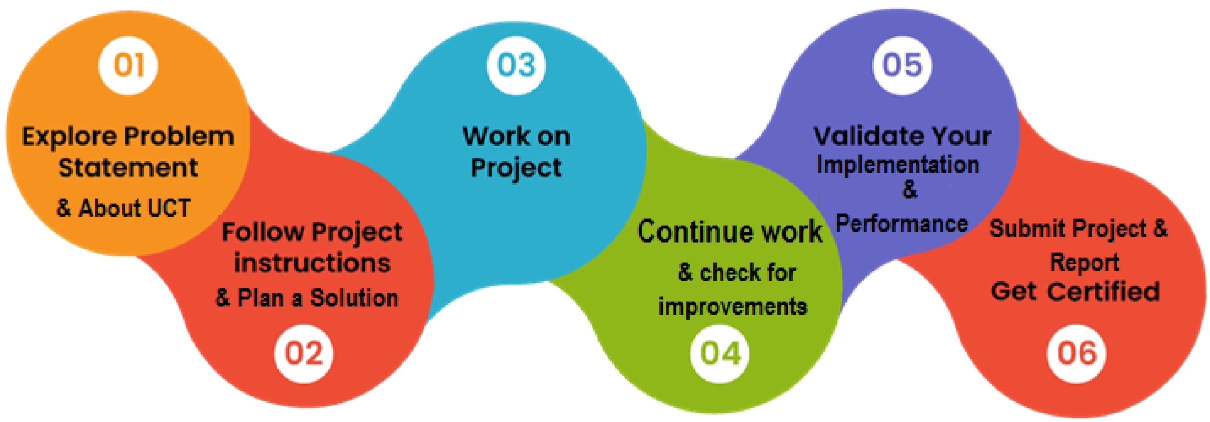
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all (with names), who have helped you directly or indirectly.

Your message to your juniors and peers.

1 Introduction

1.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

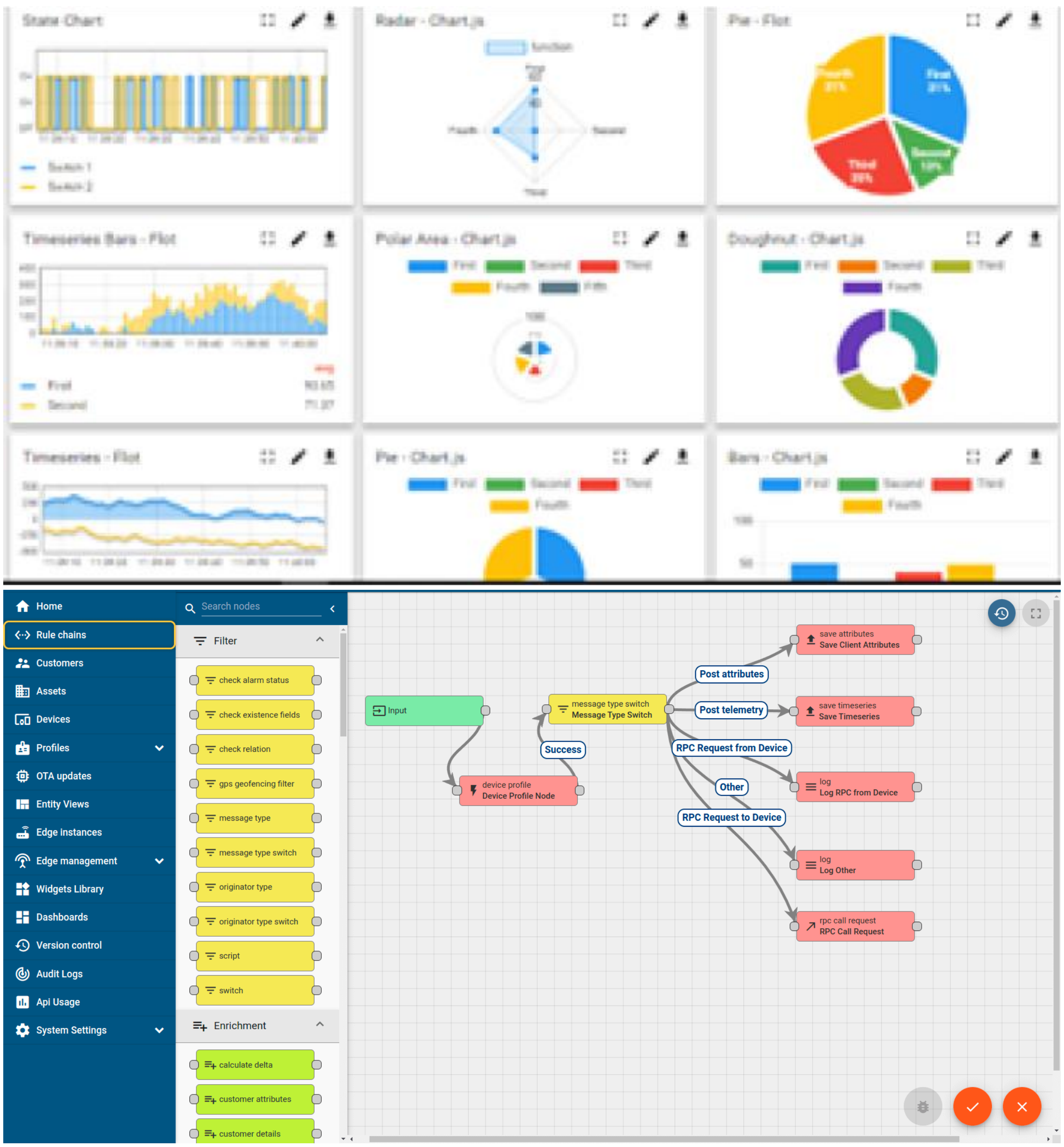
i. UCT IoT Platform (uct Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



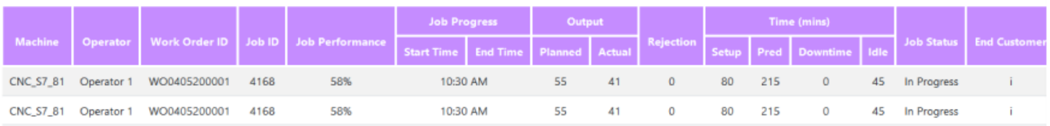
ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



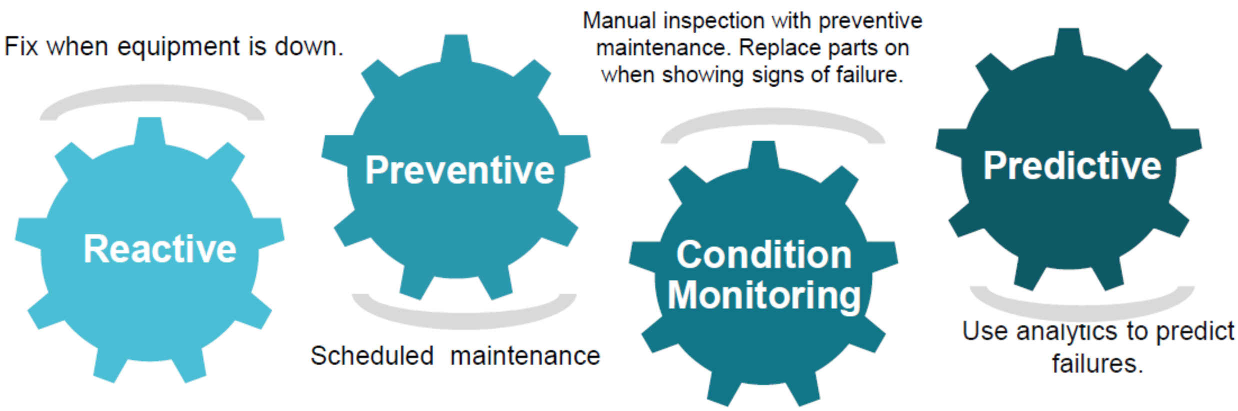
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.

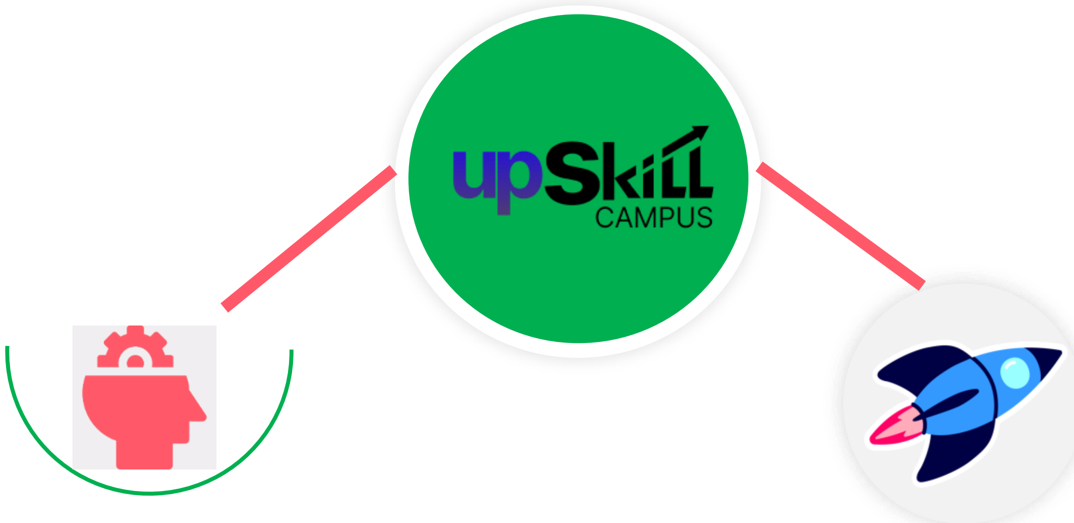


1.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.

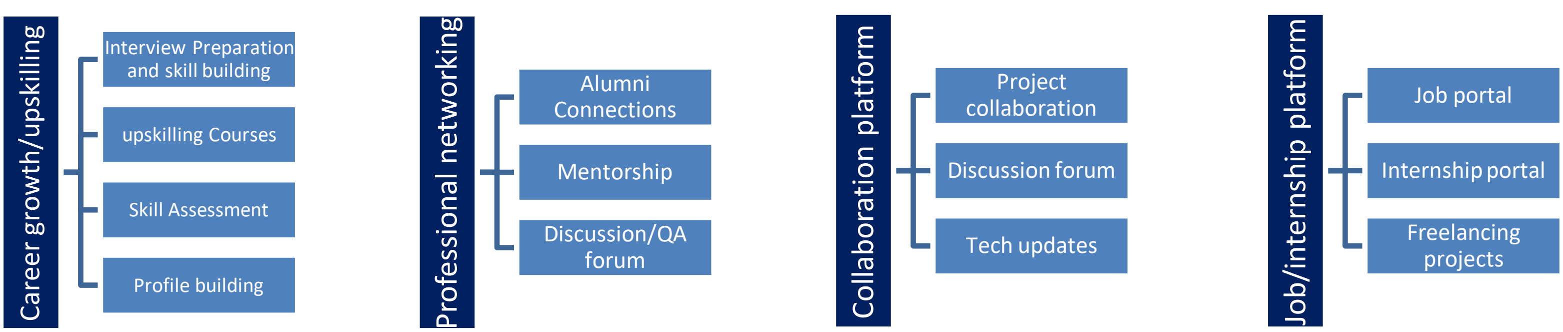
[Your College Logo]



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



1.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

1.4 Objectives of this Internship program

The objective for this internship program was to

☛ get practical experience of working in the industry.

☛ to solve real world problems.

☛ to have improved job prospects.

☛ to have Improved understanding of our field and its applications.

☛ to have Personal growth like better communication and problem solving.

1.5 Reference

[1] Python Document

[2] Pandas Document

[3] Upskill campus Learning Material

1.6 Glossary

Terms	Acronym
content Management System	CMS
Integrated Development Environment	IDE
Create, Read, Update, Delete	CRUD
Database Management System	DBMS
Structured Query Language	SQL
Command Line Interface	CLI
Graphical User Interface	GUI
User Interface	UI
Application Programming Interface	API
HyperText Transfer Protocol	HTTP
HyperText Markup Language	HTML
ascading Style Sheets	ASS
Uniform Resource Locator	URL
Software Requirements Specification	SRS

2 Problem Statement

In the assigned problem statement

The aim of this project is to develop a **Content Management System (CMS) for a Blog** that allows users to create, edit, delete, and publish blog posts easily. The system provides a user-friendly interface for managing blog content, stores data securely in a database, and displays published posts dynamically to visitors, making blog management simple, efficient, and accessible without requiring advanced technical knowledge.

2.Existing and Proposed solution

Provide summary of existing solutions provided by others, what are their limitations?

What is your proposed solution?

What value addition are you planning?

2.1 Code submission (Github link):

https://github.com/karunasrinivas56-oss/Up-skill-.git

2.2 Report submission (Github link) :

karunasrinivas56-oss/Up-skill-

3 Proposed Design/ Model

The proposed Content Management System (CMS) is designed to provide a simple and efficient platform for creating, editing, publishing, and managing blog posts. The system consists of two main modules: the Admin module and the User module. The admin manages blog content, while users can browse and read published blogs. The system uses Python for backend processing, SQLite as the database, and a graphical interface for interaction. The workflow begins with user authentication, followed by blog management operations, and finally displays the published content to readers.

3.1 High Level Diagram (if applicable)

```
          |    Admin/User    |
                   |
          |   User Interface   |
                   |
            CMS Application
            (Python Backend)
                   |
      +-----------------+-----------------+
      |                                   |
      v                                   v
+-------------------+              +-------------------+
| Blog Management |            | User Management |
 | (Add/Edit/Delete) |             | Login/Register |
 +---------------------+             +-------------------+
           \                                 /
            \                              /
              \                         /
                 \                   /
```

```
            v
    +----------------------+
   |  SQLite Database    |
    +----------------------+
              |
              v
  +----------------------+
 | Display Blog Posts   |
   +----------------------+
```

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

3.2 Low Level Diagram (if applicable)

```
      Start
        |
        v
    User Login
        |
   +--------+--------+
   |            |
Valid User?      Invalid
   |            |
  Yes        Error Message
```

```
      |
      v
  Dashboard
      |
  +------+------+------+------+
  |          |            |
  v          v            v
Add Blog   Edit Blog   Delete Blog
  |          |              |
  +------+------+-------------+
          |
  Save Changes
      |
  SQLite Database
      |
  Retrieve Blog Data
      |
  Display Blog Posts
      |
     End
```

3.3 Interfaces (if applicable)

Update with Block Diagrams, Data flow, protocols, FLOW Charts, State Machines, Memory Buffer Management.

| User |

|

| Login Interface |

|

Dashboard

|

| Add | Edit | Delete | View |

|

SQLite Database

|

Blog Display

Flow Chart

Start

|

Login

|

Authentication

|

Login Valid?

|

Yes

|

Dashboard

|

Select Operation

(Add/Edit/Delete/View)

|

Update Database

|

Display Updated Blogs

|

Logout

|

End

Protocol	Purpose
HTTP	Communication between client and application
HTTPS	Secure data transmission (if deployed online)
SQLite	Database storage for blog information

4 Memory Buffer Management

- User input is temporarily stored in memory during blog creation or editing.
- Data is validated before insertion into the SQLite database.
- After successful database operations, temporary memory is cleared to optimize performance.
- Retrieved blog records are loaded into memory only when requested for display.

This content is suitable for a **Python-based Content Management System for Blog Management** project report and fits the required **Chapter 3 (Proposed Design/Model)** format.

5 Performance Test

Performance testing was conducted to evaluate the efficiency, reliability, and usability of the Content Management System (CMS). The system was tested under different scenarios such as user authentication, blog management operations, database access, and content retrieval. The objective was to ensure that the application performs efficiently with minimal response time while maintaining data accuracy and integrity.

The major constraints identified were database performance, response time, memory usage, data consistency, and scalability. SQLite was selected as the database because it is lightweight and suitable for small to medium-scale applications. Input validation was implemented to prevent invalid data, and optimized SQL queries were used to improve execution speed.

5.1 Test Plan/ Test Cases

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	User Login with valid credentials	User logs in successfully	Successful	Pass
TC-02	User Login with invalid credentials	Error message displayed	Error displayed	Pass
TC-03	Add a new blog	Blog stored in database	Blog added successfully	Pass
TC-04	Edit existing blog	Blog updated correctly	Updated successfully	Pass
TC-05	Delete blog	Blog removed from database	Deleted successfully	Pass
TC-06	View all blogs	All published blogs displayed	Displayed correctly	Pass
TC-07	Empty blog title/content	Validation message shown	Validation successful	Pass
TC-08	Multiple blog operations	System responds without errors	Successful	Pass

5.2 Test Procedure

1. Launch the CMS application.
2. Login using valid and invalid user credentials.
3. Create a new blog post by entering the title, content, and author details.
4. Save the blog and verify that it is stored in the SQLite database.
5. Edit an existing blog and verify that the changes are updated correctly.
6. Delete a selected blog and ensure it is removed from the database.
7. View all available blog posts and verify the displayed information.
8. Test invalid inputs such as empty fields and duplicate entries.
9. Observe the application's response time, memory usage, and database operations during testing.
10. Record the results and compare them with the expected outcomes.

5.3 Performance Outcome

The Content Management System performed successfully during testing. All core functionalities, including login, blog creation, editing, deletion, and viewing, worked as expected. The average response time for database operations was less than **1 second**, and the application consumed minimal system memory due to the use of SQLite. Input validation ensured data accuracy and prevented invalid entries. The system remained stable during repeated CRUD operations without data loss or crashes.

Constraint	Impact	Solution Implemented
Database Performance	Slow retrieval with large data	Optimized SQLite queries and indexing
Response Time	Delay during operations	Lightweight backend and efficient CRUD functions
Memory Usage	Increased RAM consumption	SQLite and efficient memory management

Constraint	Impact	Solution Implemented
Data Integrity	Incorrect or duplicate data	Input validation and database constraints
Scalability	Performance may decrease with many users	Can migrate from SQLite to MySQL/PostgreSQL in future
Security	Unauthorized access	User authentication and login validation

6 My Learning

During the development of the **Content Management System (CMS) for Blog Management**, I gained practical knowledge of full-stack application development and database management. I learned how to design a user-friendly interface, implement CRUD (Create, Read, Update, Delete) operations, and connect the application with an SQLite database. This project improved my understanding of Python programming, SQL queries, input validation, and user authentication. I also learned the importance of software design, testing, debugging, and performance optimization. Overall, the project enhanced my problem-solving skills, logical thinking, and confidence in developing real-world software applications.

7 Future work scope

The proposed **Content Management System (CMS) for Blog Management** can be enhanced with several advanced features to improve its functionality, security, and scalability. Future improvements include:

- Implement role-based access control for Admin, Editor, and Author users.
- Integrate a rich text editor with image, video, and file upload support.
- Add search, filtering, categories, and tags for easier blog navigation.
- Enable user comments, likes, and social media sharing features.
- Develop a responsive mobile application for Android and iOS.
- Migrate from SQLite to MySQL or PostgreSQL to support large-scale applications.
- Implement cloud deployment using platforms such as AWS, Azure, or Google Cloud.
- Add email notifications for new blog posts and user activities.
- Improve security by implementing two-factor authentication (2FA), password encryption, and protection against SQL injection and cross-site scripting (XSS).
- Integrate analytics to track blog views, user engagement, and content performance.
- Incorporate AI-based features such as content recommendations, grammar checking, and automatic blog summarization.
- Support multilingual content management to reach users from different regions.